

도커와 배포

김미수

전남대학교 AI융합대학 인공지능학부

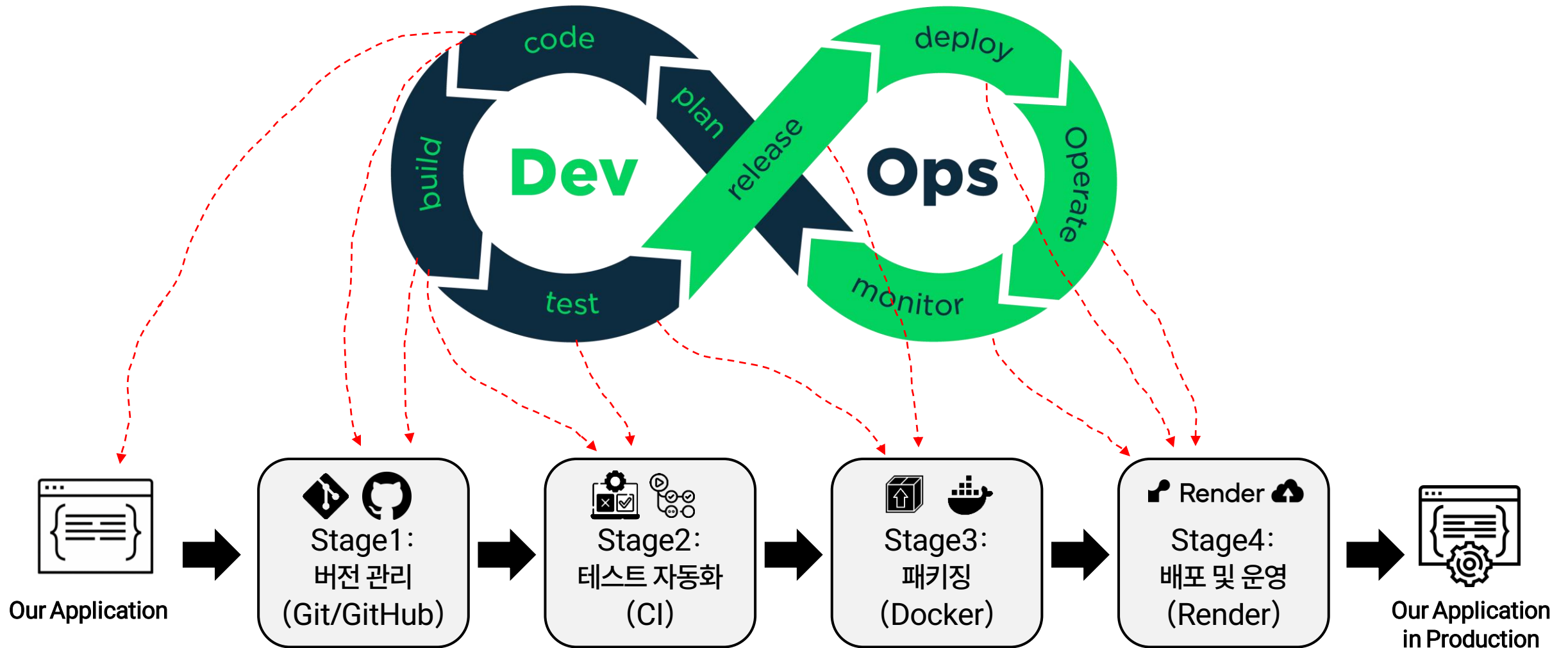
misoo.kim@jnu.ac.kr

<https://sites.google.com/view/semi-jnu>

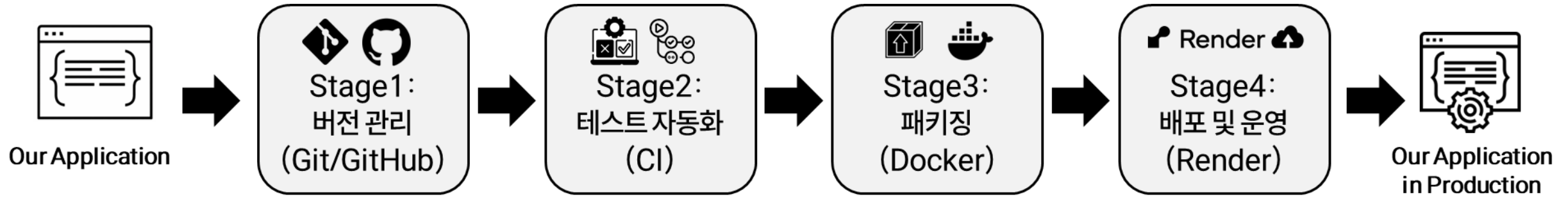
커리큘럼 변경

주차	분류	화요일 – 이론/자동화 연구들	목요일 – 실습	숙제/프로젝트
1		OT	DevOps 개요	
2	DevOps-개발	Git 기반 버전 관리	업데이트 SW + 기능 추가	실습 보고서1
3		Git 기반 브랜치 관리	브랜치와 머지	실습 보고서2
4		GitHub와 협업	Pull Request, 이슈 해결	실습 보고서3
5		테스트 자동화와 CI	Pytest, GitHub Actions	실습 보고서4
6	DevOps-운영	도커와 배포	Render, Docker	실습 보고서5
7		운영과 로그	Logging	실습 보고서6
8		중간고사 기간		자체 SW와 DevOps 파이프라인 개발 & 보고서
9	MLOps-개발	MLFlow 개요	MLFlow – 로컬	실습 보고서7
10		어린이날 (휴강)	MLFlow 실습	
11		MLFlow – 서버	MLFlow 실습	실습 보고서8
12	MLOps-운영	모델 배포와 모니터링	Render	실습 보고서9
13		모델 모니터링 및 재학습	Render	실습 보고서10
14	정리	파이프라인 복습	DevOps/MLOps 변형들	
15		기말고사 기간		자체 AISW와 MLOps 파이프라인 개발 & 보고서

수업에서 진행할 파이프라인

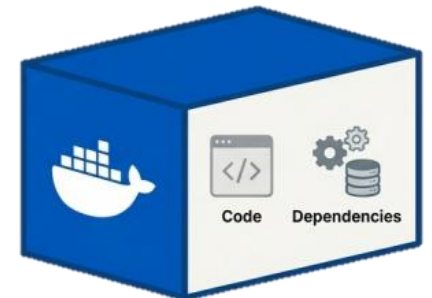


DevOps 파이프라인

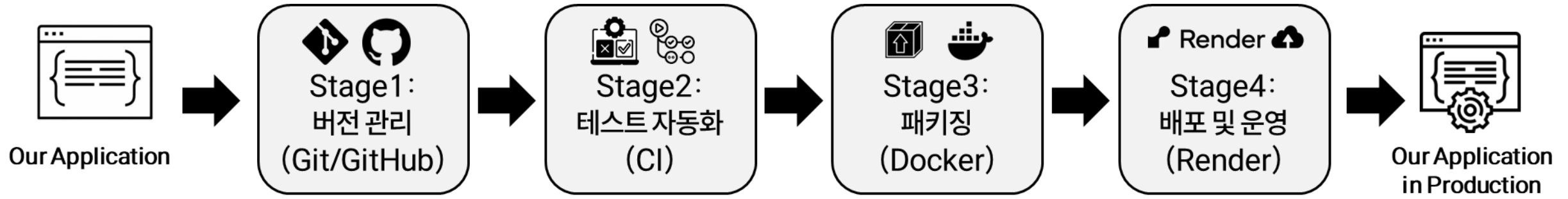


- 2단계: 일관성을 위한 패키징: “내 컴퓨터에서는 되는데...” 문제 해결

- 애플리케이션 + 실행 환경 (라이브러리, 종속성 등)을 하나의 ‘컨테이너’로 패키징
 - ▶ 이를 통해 개발, 테스트, 운영환경 어디서든 동일하게 실행되는 환경 구축
- 도구: GitHub Action 기반 Docker 이미지 빌드



DevOps 파이프라인



- 3단계: 배포와 운영 – 서비스하고 살아있는지/문제없는지 지켜보기

- 배포: 컨테이너화된 애플리케이션을 배포하여 누구나 접근할 수 있게 하는 작업
- 모니터링: 애플리케이션/운영 로그를 분석하여 문제 확인 후 디버깅
- 사용 도구: Render - 최소한의 배포 환경으로 간단함
 - GCP + IaC (Infrastructure as Code) : 복잡함, 수업 X

지금까지 방식의 한계

- 여태까지 한 것: FastAPI 기반 웹 서비스를 Render에 배포 → 사용자에게 제공

- 문제 상황: 내 PC에서는 되는데 서버에서 잘 안되는 경우가 비일비재!

- 관련 패키지가 부재

혹은 의존성 충돌

- 파이썬 버전 문제

- 그 외 OS 문제

- 등등..

```
fastapi
uvicorn
python-multipart
```



내 PC에는
Fastapi 1.1버전이
설치되어 있었고,
잘 동작함



Fastapi가 1.2버전 설치

 render 

```
match x:
    case 1:
        print("one")
```



내 PC는
3.11이라
잘 동작함



Default = Python 3.9

 render 

```
import os
print(os.getenv("MY_ENV"))
```



Windows기반
환경변수로 개발
& 동작도 잘됨



Default = Linux

 render 

Docker를 쓰면?

- 내 로컬에서 이미 설치된 상태로 가져간다!

- 기존: 코드 + requirements.txt ▶ 서버에서 설치

- Docker:

Dockerfile

→ 이미지 생성 시점에

설치 완료

▶ 완료된 것을 배포해

그대로 실행

```
fastapi
uvicorn
python-multipart
```

```
match x:
    case 1:
        print("one")
```

```
import os
print(os.getenv("MY_ENV"))
```



내 PC에는
Fastapi 1.1버전이
설치되어 있었고,
잘 동작함



내 PC는
3.11이라
잘 동작함



Windows기반
환경변수로 개발
& 동작도 잘됨



Fastapi가 1.1 설치된 환경

 render 



Python 3.11 설치된 환경

 render 



Window 환경

 render 

Docker를 쓰면?

- 내 로컬에서 이미 설치된 상태로 가져간다!
- 기존 vs 도커

구분	기존	Docker
설치 시점	배포할 때마다	이미지 만들 때 1번
환경	서버마다 다름	고정
재현성	낮음	매우 높음

Docker와 컨테이너

- 컨테이너란?

- 가상환경에서 애플리케이션과 그 종속성을 포함하여 실행할 수 있도록 만든 것
- 라이브러리, 패키지 등 목적에 맞게 환경을 세팅해 둔 것
- 동시에 나의 시스템으로부터 격리가 되어 있는 가상 공간
 - 컨테이너에 설치한 것은 컨테이너에만 있고, 로컬에는 없음

- 컨테이너화 도구: Docker, Kubernetes

- Docker: 컨테이너를 쉽게 만들고, 관리하고, 배포하도록 지원
- Kubernetes: 여러 컨테이너를 관리하는 오케스트레이션 시스템

- 이미지: 실행가능한 앱 패키지 정보 (레시피) ► 컨테이너: 이미지를 기반으로 생성됨

Docker와 컨테이너 vs 가상머신

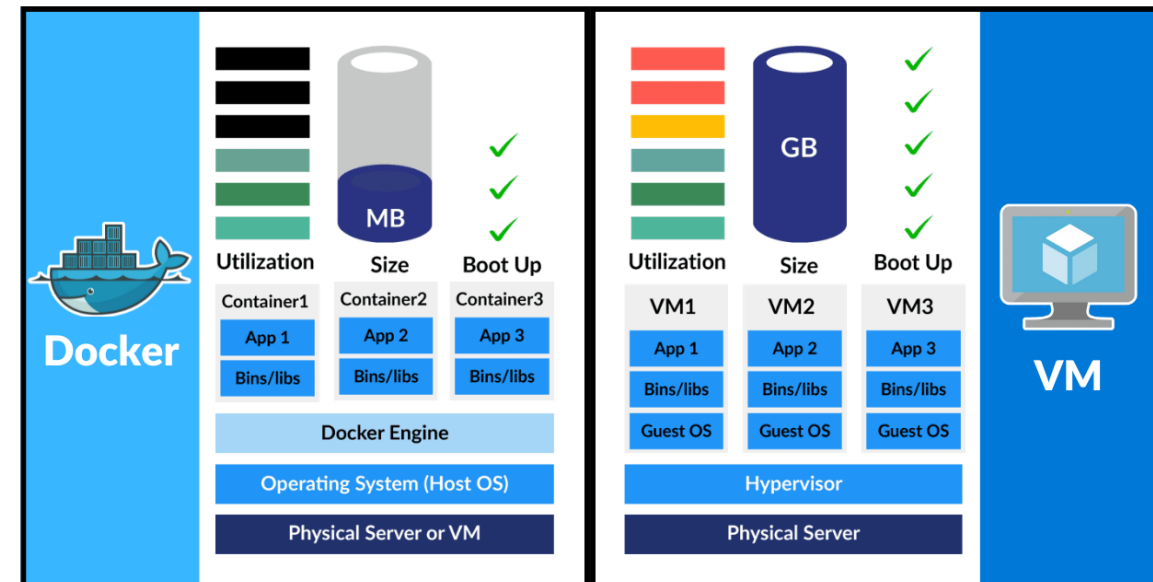
- 가상머신: 컴퓨터 안에 또다른 컴퓨터

- 단일 컴퓨터 안에 소프트웨어를 기반으로 또다른 컴퓨터가 여러 개 안에 있을 수 있도록 해주는 것
- 소프트웨어를 기반으로 “가상”으로 만들어진 공간 (머신)

- 가상환경: SW 필요한 환경만 가상화

- 하나의 OS 커널을 공유하며 독립적인 환경 제공

항목	가상머신(VM)	컨테이너(Container)
운영체제	각 VM마다 별도의 OS 실행	호스트 OS의 커널을 공유
격리성	완전한 격리 (보안성이 높음)	프로세스 단위 격리 (경량)
부팅 속도	느림 (수 분)	매우 빠름 (수 초)
리소스 사용량	높음 (CPU, RAM 많이 사용)	낮음 (가벼움)
배포 및 이동성	OS별로 환경 설정 필요	어느 환경에서나 실행 가능
주요 사용 사례	보안이 중요한 환경 여러 OS가 필요한 경우	CI/CD, 마이크로서비스, 클라우드 환경



가상환경 - 컨테이너

- 왜 요즘 다 쓸까? = 환경 통일!
- 상황 1. Python 실행 환경이 달라서 애플리케이션이 실행되지 않는 경우
 - A 개발자: Python 3.10 vs. B 개발자: Python 3.6 vs. 운영 환경: Python 3.8
 - 과거: requirements.txt 공유로 필요한 종속성 제공 후 설치
 - 문제: B 개발자와 운영 환경에 Python 버전을 수동으로 맞춰줘야 함
 - 도커 활용: Python 3.10이 포함된 컨테이너 생성 및 실행

```
docker run -it python:3.10 bash
```

≡ requirements.txt ×

```
1 pip==22.3.1
2 wheel==0.38.4
3 Pillow==9.5.0
4 setuptools==65.5.1
5 packaging==23.1
6 numpy==1.26.0
```

```
# 개발 환경 (Python 3.10)
$ python --version
Python 3.10.2
$ pip install -r requirements.txt
```

```
# 운영 서버 (Python 3.6)
$ python --version
Python 3.6.9
$ pip install -r requirements.txt
ERROR: Package 'X' requires Python >= 3.9
```

가상환경 - 컨테이너

- 상황 2. 시스템 라이브러리가 필요할 때 (pip로 해결 불가)

- A 개발자: OpenCV 활용 ▶ B 개발자: pip install opencv-python으로 설치 시도

```
ERROR: Could not find a version that satisfies the requirement opencv-python
```

- OpenCV: Python 라이브러리 뿐만 아니라 시스템 패키지(libopencv-dev)/Cmake 추가 빌드 필요
- B 개발자: apt-get install libopencv-dev 실행 (Ubuntu)
choco install opencv (Windows), brew install opencv (Mac) ▶ OS 마다 설치 다름
- 도커 활용: 필요한 OS 패키지를 포함해서 어디서든 동일 환경 제공

dockerfile

```
FROM python:3.10
RUN apt-get update && apt-get install -y libopencv-dev
RUN pip install opencv-python
```

```
docker build -t my-opencv-app .
docker run my-opencv-app
```

가상환경 - 도커

- 얼마나 많이 쓰이나?
 - <https://aws.amazon.com/ko/docker/>
 - <https://github.com/WordPress/wordpress-develop>
 - 과거: 코드만 오픈 ► 환경/종속성 문제로 제대로 실행해볼 수 없음
 - 현재: Docker 파일 함께 제공!
- Docker Hub: 도커 저장소
 - <https://hub.docker.com/>

실습 10. Docker와 배포

도커란?

- 패키징: 코드 + 실행에 필요한 모든 환경을 하나로 묶는 것
- 기존: requirement 기반 패키징 (불완전. Os/python version/sys lib 정보 없음)
- 도커: 실행 환경을 코드로 고정해 로컬/서버 어디서든 똑같이 돌리게 하는 패키징 도구
 - Dockerfile: 이미지를 만드는 레시피 (RUN pip install, CMD uvicorn...)
 - 빌드: Dockerfile을 기반으로 이미지를 만드는 과정 (docker build -t spamcheck .)
 - 이미지: 실행 가능한 환경의 설계도, 불변, 빌드 결과물, 실행 전 상태 (spamcheck:local)
 - 실행: 이미지를 기반으로 컨테이너를 실행 (docker run -p 10000:10000 spamcheck)
 - 컨테이너: 이미지를 실제로 실행한 것, 이미지 → 실행 → 컨테이너 (docker run spamcheck:local)
 - 가상 머신이 아님! 가상 환경!

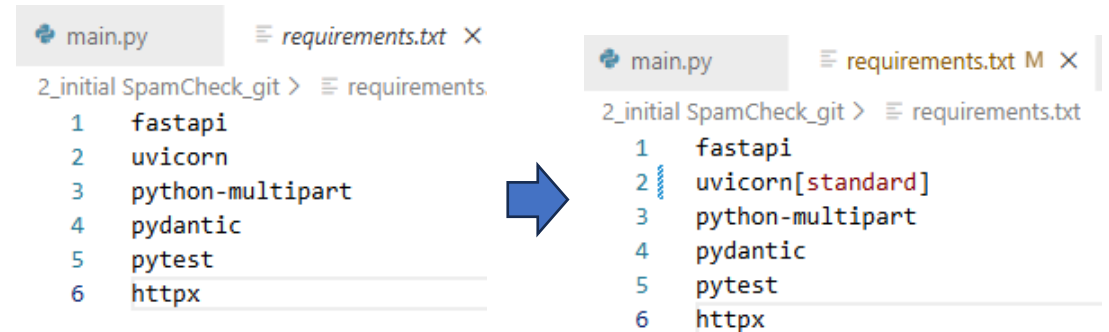
도커란?

- 왜 해야 하는가?
- 개발자 A: Window 환경에서 실행 됨
- 신규 개발자 B: Linux에서 실행 안됨

= 두 개의 차이가 발생하여 에러가 생길 수 있고, 결과가 달라질 수 있음

= 공통된 환경 필수

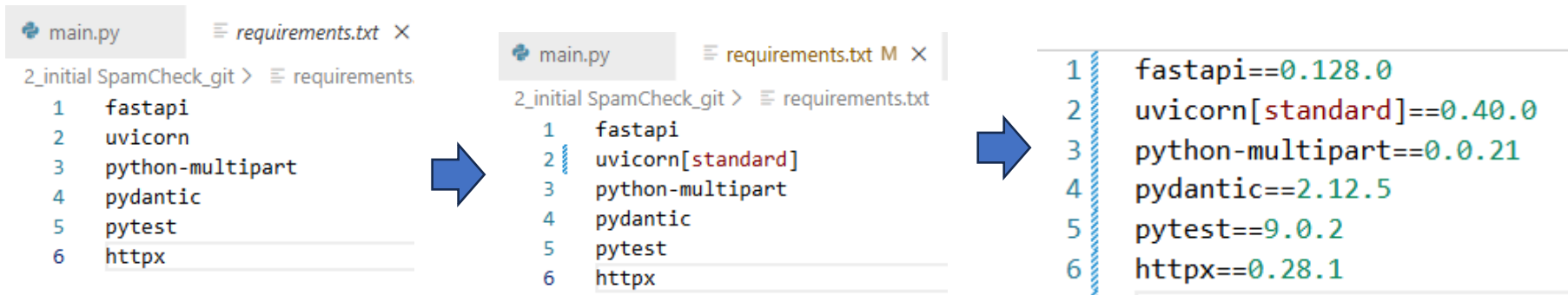
- 준비사항 = Docker 설치하고 시작!
- uvicorn[standard] 추가 (추가 의존성 묶음 모두)
- `pip install -r requirements.txt`



도커란?

- 버전 명시한 requirements.txt로 확장
 - `conda list | findstr "fastapi uvicorn python-multipart pydantic pytest httpx"`

fastapi	0.128.0	pypi_0	pypi
httpx	0.28.1	pypi_0	pypi
pydantic	2.12.5	pypi_0	pypi
pydantic-core	2.41.5	pypi_0	pypi
pytest	9.0.2	pypi_0	pypi
python-multipart	0.0.21	pypi_0	pypi
uvicorn	0.40.0	pypi_0	pypi



도커화 준비

- Requirement 수정 : uvicorn[standard]
- .dockerignore과 Dockerfile 만들기

```
__pycache__/  
.pytest_cache/  
.venv/  
venv/  
.git/  
.github/  
.env  
*.pyc
```

```
2_initial SpamCheck_git  
├── __pycache__  
├── .github  
├── .pytest_cache  
├── app  
├── static  
├── tests  
├── .dockerignore  
├── .gitignore  
├── Dockerfile  
└── requirements.txt
```

```
# 로컬이 3.13이라. 이전 버전이면 거기에 맞춰서 수정 3.11가 가장 안정되었다고들 함  
FROM python:3.13-slim  
  
WORKDIR /app  
  
# 의존성 설치  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
# 소스 복사  
COPY . .  
  
# Render에서 PORT를 주고, 로컬에서는 10000 기본값  
ENV PORT=10000  
EXPOSE 10000  
  
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT}"]
```

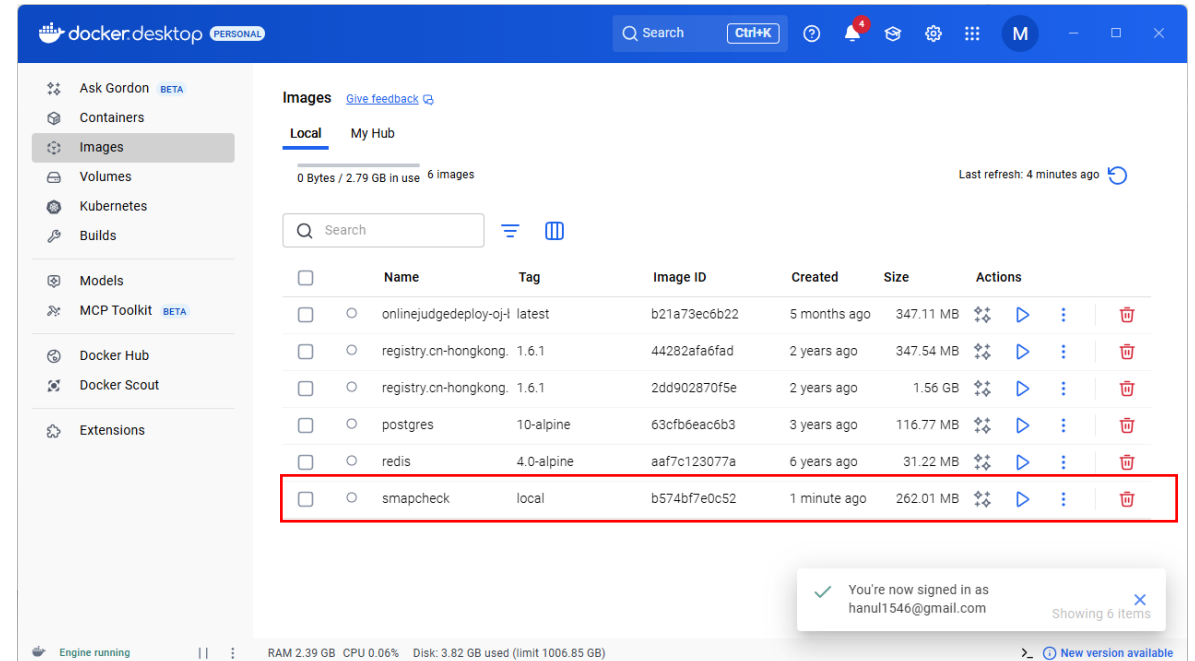
도커화 준비

- 도커 빌드: `docker build -t spamcheck:local .`



```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>docker build -t smapcheck:local .
```

```
[+] Building 18.0s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 489B
=> [internal] load metadata for docker.io/library/python:3.13-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 115B
=> [1/5] FROM docker.io/library/python:3.13-slim@sha256:3de9a8d7aedbb7984dc18f2dff178a7850f16c1ae7c34ba9d7ecc23d0755e35f
=> => resolve docker.io/library/python:3.13-slim@sha256:3de9a8d7aedbb7984dc18f2dff178a7850f16c1ae7c34ba9d7ecc23d0755e35f
=> => sha256:f1cadbd7abd229d3d8c50b4aa381724025f6bfe89783a8d2bfd6fa751a75946b 252B / 252B
=> => sha256:686599c79c8709aa5d9f1abf19c75b1760ae0a0ea0335206fe1db9a8793e09f6 11.80MB / 11.80MB
=> => sha256:03af238a5946948d06e8485bb27b05831c5d13f0b3781a01fe347aaf847c2400 1.29MB / 1.29MB
=> => sha256:0c8d55a45c0dc58de60579b9cc5b708de9e7957f4591fc7de941b67c7e245da0 29.78MB / 29.78MB
=> => extracting sha256:0c8d55a45c0dc58de60579b9cc5b708de9e7957f4591fc7de941b67c7e245da0
=> => extracting sha256:03af238a5946948d06e8485bb27b05831c5d13f0b3781a01fe347aaf847c2400
=> => extracting sha256:686599c79c8709aa5d9f1abf19c75b1760ae0a0ea0335206fe1db9a8793e09f6
=> => extracting sha256:f1cadbd7abd229d3d8c50b4aa381724025f6bfe89783a8d2bfd6fa751a75946b
=> [internal] load build context
=> => transferring context: 12.63kB
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:e1b26625a0c7583b7a0dee143975cbeecf89a0b1b41292fe4e5173654c31d885
=> => exporting config sha256:9c495974f0be1468269d163fc982b0f22ce232b7deb2beaf0840086c4c4d7cc
=> => exporting attestation manifest sha256:a7059b8e140bf5353af02cccd873a82a88e165e190f26b20d13ca4c2de5ab0f46
=> => exporting manifest list sha256:b574bf7e0c52b3596573aa692032d60236438c7350c0653f97837cbff9337454
=> => naming to docker.io/library/smapcheck:local
=> => unpacking to docker.io/library/smapcheck:local
```



도커화 준비

- 도커 실행: `docker run --rm -p 10000:10000 spamcheck:local`

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>docker run --rm -p 10000:10000 spamcheck:local
INFO:      Started server process [7]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:10000 (Press CTRL+C to quit)
^
```

☐	●	spamcheck	local	32e6d2ae741e	2 minutes ago	262.01 MB				
--------------------------	--------------------------------------	-----------	-------	--------------	---------------	-----------	--	--	--	--

Containers [Give feedback](#)

Container CPU usage

0.08% / 3200% (32 CPUs available)

Container memory usage

36MB / 61.19GB

[Show charts](#)

Search



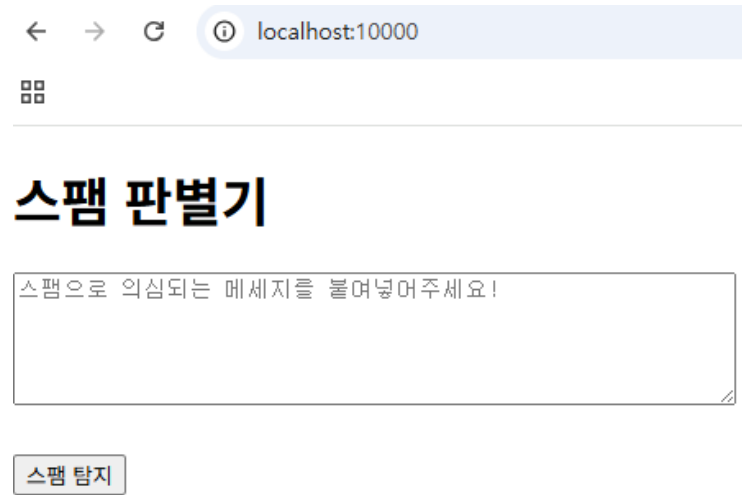
Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	● clever_einstein	f524f4f77d40	spamcheck:local	10000:10000	0.08%	1 minute ago	

도커화 준비

- 도커 실행 확인

- <http://localhost:10000/>
- <http://localhost:10000/docs>



SpamCheck Web 0.1.0 OAS 3.1
/openapi.json

default

GET

/ Home

▼

POST

/classify Classify

▼

Containers / clever_einstein

clever_einstein

f524f4f77d40 spamcheck:local 10000:10000

logs Inspect Bind mounts Exec Files Stats

INFO: Started server process [7]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:10000 (Press CTRL+C to quit)
INFO: 172.17.0.1:58948 - "GET / HTTP/1.1" 200 OK
INFO: 172.17.0.1:58948 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 172.17.0.1:57128 - "GET / HTTP/1.1" 200 OK
INFO: 172.17.0.1:48012 - "POST /classify HTTP/1.1" 200 OK
INFO: 172.17.0.1:48012 - "GET /docs HTTP/1.1" 200 OK
INFO: 172.17.0.1:48012 - "GET /openapi.json HTTP/1.1" 200 OK

Containers [Give feedback](#)

Container CPU usage 0.08% / 3200% (32 CPUs available)

Container memory usage 36MB / 61.19GB

[Show charts](#)

Search		Only show running containers						
	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions	
☐	clever_einstein	f524f4f77d40	spamcheck:local	10000:10000	0.08%	1 minute ago	  	

도커 배포

- GitHub에 푸시

- git add .
- git commit -m "Dockerize for deployment"
- git push origin main

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git add .
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git commit -m "Dockerize for deployment"
[main 33bd09d] Dockerize for deployment
3 files changed, 26 insertions(+), 1 deletion(-)
create mode 100644 .dockerignore
create mode 100644 Dockerfile
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 32 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 959 bytes | 959.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/MisooKim/jnu2603-spamchecker_test.git
39df28f..33bd09d main -> main
```

 jnu2603-spamchecker_test Private

 main

 1 Branch  0 Tags

 MisooKim Dockerize for deployment ✓

 .github/workflows

Use Gi

WEB SERVICE

jnu2603-spamchecker_test

Connect

Manual Deploy

Python 3 Free Upgrade your instance →

Service ID: srv-d65vclmsb7us73aqo9ag

MisooKim / jnu2603-spamchecker_test main

<https://jnu2603-spamchecker-test.onrender.com>

① Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

[Upgrade now](#)

Filter events 31

Deploy started for 33bd09d: Dockerize for deployment



New commit via Auto-Deploy

February 12, 2026 at 2:20 PM



Cancel deploy



Deploy live for 39df28f: fix 에러발생

February 11, 2026 at 12:58 PM

도커 배포

• GitHub에 푸시

- git add .
- git commit -m "Dockerize for deployment"
- git push origin main

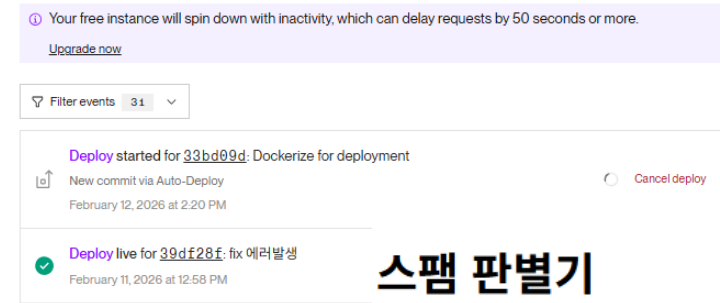
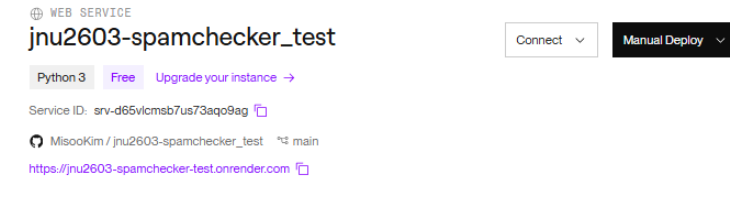
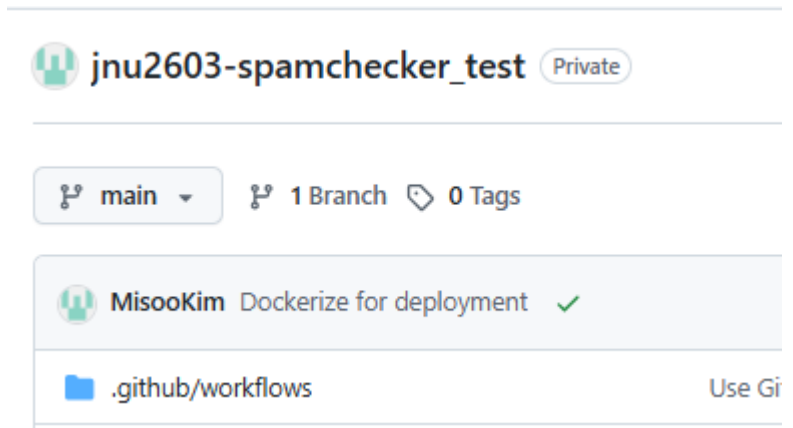
```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git add .
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git commit -m "Dockerize for deployment"
[main 33bd09d] Dockerize for deployment
3 files changed, 26 insertions(+), 1 deletion(-)
create mode 100644 .dockerignore
create mode 100644 Dockerfile
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 32 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 959 bytes | 959.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/MisooKim/jnu2603-spamchecker_test.git
39df28f..33bd09d main -> main
```

▶ 기존 render 세팅이 python 기반임 → 도커로 배포된 것이 X

- 그냥 docker 파일만 올라간 상태



스팸 판별기

Hey!

스팸 탐지

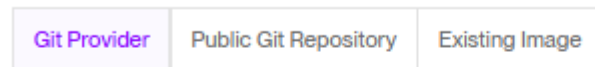
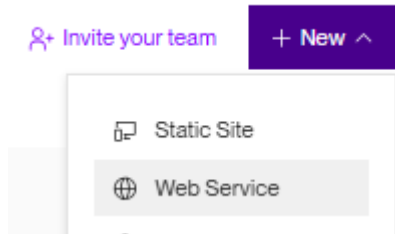
결과: ham (score=0)

도커로 배포하게 변경하기

- 기존 배포 서비스 삭제 후 docker로 재생성 (안해도 됨)

- Setting → delete

- Docker 배포로 변경



MisooKim / jnu2603-spamchecker_test 9m ago

Name
A unique name for your web service.

jnu2603-spamchecker_test

Project Optional
Add this web service to a [project](#) once it's created.

 **Create a new project to add this to?**
You don't have any projects in this workspace. resources into environments so you can better

[+ Create a project](#)

Language
Choose the [runtime environment](#) for this service.

Docker

Branch
The Git branch to build and deploy.

main

For hobby projects

Free \$0 / month	512 MB (RAM) 0.1 CPU
----------------------------	-------------------------

For professional use
For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime
- SSH Access
- Scaling
- One-off jobs
- Support for persistent disks

Starter \$7 / month	512 MB (RAM) 0.5 CPU
Pro \$85 / month	4 GB (RAM) 2 CPU
Pro Max \$225 / month	16 GB (RAM) 4 CPU

[Need a custom instance type?](#) We sup

Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

NAME_OF_VARIABLE	value
------------------	-------

[+ Add Environment Variable](#) [Add from .env](#)

Advanced

[Deploy Web Service](#)

도커로 배포하게 변경하기

• 도커 배포 완료!

```
Feb 12
02:29:33 PM ==> Cloning from https://github.com/MisooKim/jnu2603-spamchecker_test
02:29:34 PM ==> Checking out commit 33bd09d0f708647a33adc8e3a1215a7efe5c4c2e in branch main
02:30:02 PM #1 [internal] load build definition from Dockerfile
02:30:02 PM #1 transferring dockerfile: 472B done
02:30:02 PM #1 DONE 0.0s
02:30:02 PM
02:30:02 PM #2 [internal] load metadata for docker.io/library/python:3.13-slim
02:30:02 PM #2 ...
02:30:02 PM
02:30:02 PM #3 [auth] library/python:pull render-prod/docker-mirror-repository/library/python:pull
token for us-west1-docker.pkg.dev
02:30:02 PM #3 DONE 0.0s
02:30:02 PM
02:30:02 PM #2 [internal] load metadata for docker.io/library/python:3.13-slim
02:30:02 PM #2 DONE 0.6s
02:30:02 PM
02:30:02 PM #4 [internal] load .dockerignore
02:30:02 PM #4 transferring context: 107B done
02:30:02 PM #4 DONE 0.0s
```

```
02:30:54 PM #9 20.12 Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0 anyio-
4.12.1 certifi-2020.1.4 click-8.3.1 fastapi-0.128.8 h11-0.16.0 httpcore-1.0.9
httpx-0.28.1 idna-3.11 iniconfig-2.3.0 packaging-26.0 pluggy-1.6.0
pydantic-2.12.5 pydantic-core-2.41.5 pygments-2.19.2 pytest-9.0.2 python-dotenv-1.2.1
python-multipart-0.0.22 pyyaml-6.0.3 starlette-0.52.1 typing-extensions-4.15.0
typing-inspection-0.4.2 uvicorn-0.40.0 uvloop-0.22.1 watchfiles-1.1.1 websockets-16.0
```

```
02:31:19 PM #12 DONE 0.5s
02:31:41 PM Pushing image to registry...
02:31:44 PM Upload succeeded
02:32:28 PM ==> Deploying...
02:32:28 PM ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
```

```
Render
14:34:47 INCOMING HTTP REQUEST DETECTED ...
14:34:50 SERVICE WAKING UP ...

14:34:54 ALLOCATING COMPUTE RESOURCES ...
14:34:57 PREPARING INSTANCE FOR INITIALIZATION ...
14:35:01 STARTING THE INSTANCE ...
14:35:07 ENVIRONMENT VARIABLES INJECTED ...
14:35:09 FINALIZING STARTUP ...
14:35:11 OPTIMIZING DEPLOYMENT ...
14:35:13 STEADY HANDS. CLEAN LOGS. YOUR APP IS ALMOST LIVE ...
```

```
instance
02:40:10 PM [wpz12] INFO: Started server process [7]
02:40:10 PM [wpz12] INFO: Waiting for application startup.
02:40:10 PM [wpz12] INFO: Application startup complete.
02:40:10 PM [wpz12] INFO: Uvicorn running on http://0.0.0.0:10000 (Press CTRL+C to quit)
02:40:11 PM [wpz12] INFO: 127.0.0.1:49118 - "HEAD / HTTP/1.1" 405 Method Not Allowed
02:40:14 PM ==> Your service is live 🎉
02:40:14 PM ==>
02:40:14 PM ==>
02:40:14 PM ==> Available at your primary URL https://jnu2603-spamchecker-test.onrender.com/
02:40:14 PM ==>
02:40:14 PM ==>
02:40:14 PM [wpz12] INFO: 10.19.191.3:0 - "GET / HTTP/1.1" 200 OK
02:40:17 PM [wpz12] INFO: 10.19.235.3:0 - "HEAD / HTTP/1.1" 405 Method Not Allowed
02:40:18 PM [wpz12] INFO: 10.19.235.3:0 - "GET / HTTP/1.1" 200 OK
```

스팸 판별기

스팸으로 의심되는 메시지를 붙여넣어주세요!

스팸 탐지

← → ↺ jnu2603-spamchecker-test.onrender.com/docs



SpamCheck Web 0.1.0 OAS 3.1

/openapi.json

default

GET / Home

POST /classify Classify

실습 11. Docker CI

현재까지의 흐름

- 커밋 → Test → CI → Merge → Render에서 Docker화 후 배포
- 이슈: Render에서 Docker화가 실패하면? 서비스 다운됨
- 해결책: GitHub상에서 패키징이 제대로 되는지 확인 후 merge 되도록 하기!
- Render에서 Auto-deploy 설정 변경
 - CI check 후 배포

Auto-Deploy

By default, Render automatically deploys your service whenever you update its code or configuration. Disable to handle deploys manually.

[Learn more.](#)

After CI Checks Pass

GitHub Action 추가

- .github/workflows/docker-verify.yml
- 커밋
 - Git add .
 - Git commit -m “Add action to verify docker”
 - Git push origin main

```
name: Docker Verify
on:
  pull_request:
  push:
    branches: [ "main" ]
jobs:
  docker-verify:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t spamcheck:ci .
      - name: Run container
        run: |
          docker run -d --name spamcheck_ci \
            -e PORT=10000 \
            -p 10000:10000 \
            spamcheck:ci
      - name: Check container is running
        run: |
          sleep 3
          docker ps | grep spamcheck_ci
      - name: Cleanup
        if: always()
        run: docker rm -f spamcheck_ci || true
```

add action to verify docker
CI #8: Commit b7967ea pushed by MisooKim

add action to verify docker
Docker Verify #1: Commit b7967ea pushed by MisooKim

```
✓ Run tests
1 ▶ Run pytest
11 ===== test session starts
12 platform linux -- Python 3.10.19, pytest-9.0.2
13 rootdir: /home/runner/work/jnu2603-spamchecker
14 plugins: anyio-4.12.1
15 collected 2 items
16
17 tests/test_api.py .
18 tests/test_spam.py .
19
20 ===== 2 passed in 0.4
```

```
> ✓ Set up job
> ✓ Checkout
> ✓ Build Docker image
> ✓ Run container
✓ ✓ Check container is running
1 ▶ Run sleep 3
5 23f3d79a7ad1 spamcheck:ci "sh -c 'uvi
```

WEB SERVICE
jnu2603-spamchecker_test Docker Free
Service ID: srv-d66mac6ab7us73bfuog
MisooKim / jnu2603-spamchecker_test *1 main
<https://jnu2603-spamchecker-test.onrender.com>

① Your free instance will spin down with inactivity, which can delay re

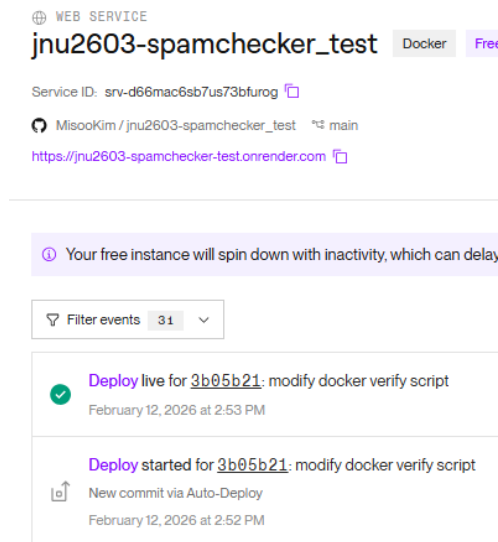
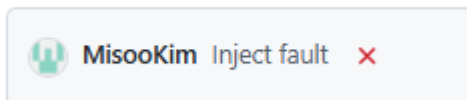
Filter events 31

✓ Deploy live for 3b05b21: modify docker verify script
February 12, 2026 at 2:53 PM

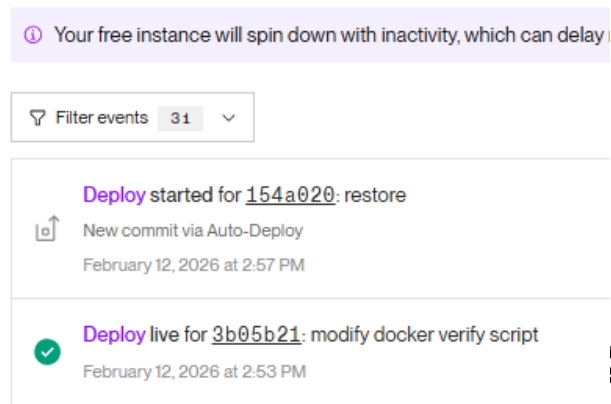
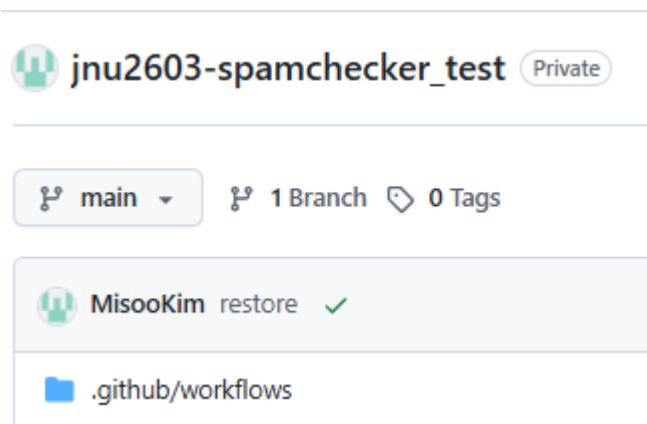
Deploy started for 3b05b21: modify docker verify script
New commit via Auto-Deploy
February 12, 2026 at 2:52 PM

GitHub Action 추가

- 일부러 결함 삽입 후 푸시하면?



- 고치고 푸시하면?



```
name: Docker Verify
on:
  pull_request:
  push:
    branches: [ "main" ]
jobs:
  docker-verify:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t spamcheck:ci .
      - name: Run container
        run: |
          docker run -d --name spamcheck_ci \
            -e PORT=10000 \
            -p 10000:10000 \
            spamcheck:ci
      - name: Check container is running
        run: |
          sleep 3
          docker ps | grep spamcheck_ci
      - name: Cleanup
        if: always()
        run: docker rm -f spamcheck_ci || true
```

숙제 6. 실습 10, 11

- 이번 주 일요일 자정까지

- 자유 양식

1. 실습 10

- 목적, 방법, 결과 스크린샷 및 설명

2. 실습 11

- 목적, 방법, 결과 스크린샷 및 설명

- 나중에 GitHub와 연동하여 이력 확인할 것이니 매주 복습 잘하기